

AI Competition — Challenge Document

Powered by IFS — Dedicated Competition Partner

Initial Round: 28 August – 9 September 2026

1. About the Competition

The AI Competition at SLIIT Codefest 2026, powered by IFS, challenges university teams to design and build working AI solutions to a realistic enterprise problem. Over a two-week initial round, teams of four will tackle one defined challenge with a choice of three sub-tracks, submit their working solutions, and compete for a place in the on-site final round and the Codefest 2026 Grand Finale.

This competition rewards genuine engineering and insight. You are free — and expected — to use AI tools in your work. What earns marks is not the polish of your output alone, but the quality of your thinking: the approach you chose, why you chose it, what you tried, what failed, and how well your solution actually works.

2. Timeline

Phase	Dates	What Happens
Launch	28 August 2026	AI awareness session, competition briefing, and release of this challenge document.
Initial Round	28 Aug – 9 Sep 2026	Teams build their solutions and submit all deliverables by 9 September 2026, 11:30 PM.
Evaluation	9 Sep – 16 Sep 2026	Judging panel reviews all submissions. The top 10 teams are shortlisted and notified.
IFS Site Visit	18 September 2026	Top 10 teams are invited to a special program at IFS premises. Final-round guidelines will be communicated.
Final Round	21 September 2026	On-site final competition, 21-22 September 2026. Finalists demonstrate and defend their solutions before the judging panel.
Grand Finale	24 September 2026	Champion, 1st Runner-up, and 2nd Runner-up announced at the Codefest 2026 Awards Ceremony.

3. The Challenge: Intelligent Document Assistant

Enterprise documents are messy. Real technical documentation mixes scanned pages, diagrams, tables, and images with text — spread across many documents and thousands of pages. The information a user needs are in there, but finding it, connecting it, and presenting it usefully is hard. Your challenge is to build an AI assistant that makes this documentation genuinely usable.

Choose ONE of the three sub-tracks below. Each is a complete challenge on its own. Ambitious teams may address more than one, but a strong, working solution to a single sub-track will always beat a shallow attempt at several.

Sub-track 1A — Rich Answers, Not Just Text

Technical manuals contain scanned images, diagrams, and tables mixed with text. Build an AI assistant that answers questions using this content — and doesn't stop at text. When a diagram, table, or image is relevant to the answer, the assistant should embed it directly in its response alongside the explanation, so the user sees the actual figure, not just a description of it.

Sub-track 1B — Connecting Facts Across Thousands of Pages

Some questions — like “which other equipment is affected if component Y fails?” — can only be answered by linking facts spread across multiple documents spanning thousands of pages. Systems that simply fetch a few relevant-looking passages retrieve only a partial picture and miss the most important facts, because no single passage contains the full answer. Build a system that can connect related information across the whole document set and answer questions with no single document answers on its own.

Sub-track 1C — Searching the Way a Human Does

Some questions can't be answered in one lookup. A human expert reads what they find, learns from it, realizes what's still missing, and goes looking for more — reasoning between each step until they have enough to answer. Build an assistant that works the same way: it decides where to look and in what order, checks whether it has enough information, and if not, uses what it has learned so far to guide the next search — repeating until it can give a complete answer.

Document Corpus — The Ashen Era Archive

All teams work against the official competition corpus: the Ashen Era Archive, a complete fantasy franchise archive created specifically for this competition. It contains four novel volumes, around ninety wiki articles, three codex data books full of tables and figure plates, roughly 150 pieces of in-world ephemera (letters, ledgers, trial transcripts, scanned pages), and a collection of images. In total, that is 415 documents and about 1,277 pages in mixed formats: PDF, DOCX, markdown, plain text, and simulated scans.

The world is entirely invented. No public AI model has ever seen it, so general knowledge will not help you: your system must genuinely work from the provided documents. The archive is deeply interconnected. The same characters, factions, events, and artifacts surface across the novels, the wiki, the codexes, and the ephemera, but no single document tells the whole story. And like any real archive, its sources differ in reliability: a tavern ballad and an official codex entry do not always agree.

A development question set (sample_questions.json, 20 questions) is included with the corpus so you can test your system. Final judging uses a separate, unpublished question set of the same style. The corpus is read-only: use it as provided, and build your solution around it.

4. Rules & Eligibility

- **Team size:** Teams consist of exactly 4 members. All members must be eligible participants of SLIIT Codefest 2026.
- **Technology freedom:** Any programming language, framework, model, or service may be used unless it violates the AI usage policy below. Paid services are permitted but never required — the challenge is fully solvable with free resources.
- **Original work:** All work must be done within the competition window (28 August – 9 September 2026). Pre-existing generic libraries are fine; pre-built solutions to this specific challenge are not.
- **Intellectual property:** Teams retain ownership of their work. By submitting, teams grant the organizers the right to review, run, and showcase submissions for competition purposes.
- **Integrity:** Plagiarism, submission of another party's work as your own, or misrepresentation of what your system can do will result in disqualification.

4.1 AI Usage Policy

AI tools (coding assistants, chat models, generators) are permitted and expected — this is an AI competition. However:

- Your submission report must include an AI Usage Disclosure: which tools you used, for what, and which decisions were made by the team. Honest disclosure is expected and carries no penalty.
- Chat history: Export the full conversation logs with any AI agents used during development (coding assistants, chat models) as plain text (.txt) files and include them in your Git repository submission.
- Submissions are judged on human insight, working functionality, and demonstrated understanding — generic, unvalidated output scores low regardless of polish.
- Every team member must be able to explain, justify, and modify any part of the submission on demand. The inability to do so in the final round will result in disqualification.

5. Submission Deliverables

All three deliverables must be submitted by 9 September 2026, 11:30 PM via the submission link provided in the email. Incomplete submissions will not be evaluated.

5.1 Demonstration Video (10 minutes maximum)

- **Live demo:** At least 3–4 minutes must be a live, unedited screen recording demonstrating the working prototype end-to-end on real inputs.

- **Presentation:** The remaining time may be used to present the problem, your solution, your architecture, and the technologies you used — slides, narration, or walkthrough, your choice.
- **Limits:** Videos over 10 minutes, or without a genuine live demonstration segment, will be penalized.
- **Hosting:** Upload the finished video to YouTube as an unlisted video (not public, not private) and paste that link at the top of your submission report (5.3). The uploaded timestamp will be checked.

5.2 Git Repository (ZIP archive)

- **Full repo with history:** Submit a ZIP of your complete Git repository including the full commit history. The commits history should reflect your team's actual working process across the two weeks.
- **Documentation:** Include README with setup and run instructions, plus markdown documentation of your design and key decisions.
- **Diagrams:** Include architecture and design diagrams (any format, embedded or linked in the markdown docs).
- **AI chat logs:** Include the exported chat history text files described in the AI Usage Policy (4.1).
- Below is the recommended folder structure, with example files.

```
<Team_Name>.zip
├── .git/
├── README.md
├── docs/
│   ├── architecture.md
│   ├── decisions.md
│   ├── limitations.md
│   └── diagrams/
├── src/
├── ai_usage/
│   ├── ai-usage-disclosure.md      # mandatory (AI chat logs)
│   ├── skills
│   ├── claude.md
│   └── context.md
├── configuration-example/
└── submission_report.pdf
```

5.3 Submission Report (PDF)

A single PDF, maximum 5 pages, containing:

- Problem statement and your chosen sub-track
- Solution overview and architecture (with diagram)
- Key technical decisions and why you made them
- What works, what doesn't, and limitations — including approaches you tried that failed

- AI Usage Disclosure
- Team member names, roles, and contributions

6. Evaluation Criteria

Submissions are scored by a judging panel using the rubric below. Each submission is reviewed independently by multiple judges.

Criterion	Weight	What Judges Look For
Technical execution & robustness	25%	Does it actually work, end-to-end, on inputs the team didn't script? Quality of engineering; how well the solution handles the hard parts of the chosen sub-track.
Problem understanding & insight	15%	Evidence the team understood the real problem and its difficulties — not a generic solution with the challenge's name on it.
Human–AI collaboration quality	15%	How well the team directed and collaborated with AI tools — not one-shot generation. We look for the team's own ideas and thought process steering the AI: iterative refinement, questioning and correcting AI output, and mixing the team's thinking and innovation with the capabilities of AI coding assistants. Evidence comes from the exported chat logs, the AI Usage Disclosure, and the commit history.
Engineering best practices	15%	Git discipline: meaningful atomic commits, clear messages, sensible history. Repository organization. Code quality: modular design with separation of concerns, docstrings and comments where they matter. Dependency and environment management (e.g., requirements/lock files, virtual environments) and reproducibility — can a judge set up and run your project from the README alone?
Technical judgment & decisions	10%	Why this approach over alternatives; honest analysis of trade-offs, failures, and limitations; human decisions clearly visible.
Impact & relevance	10%	Usefulness of the solution in a realistic enterprise setting; how convincingly it addresses the chosen sub-track.
Presentation, documentation & demonstration	10%	Clarity of the video presentation, the quality and credibility of the live demonstration, and the clarity

Criterion	Weight	What Judges Look For
		of the report, markdown documentation, and architecture diagrams.

Note: Teams might be directly contacted by judges to assess the authenticity of the submission. Generic, unvalidated ideas score low regardless of polish. One-shot AI generation — prompting a tool to produce the solution and submitting the output — scores poorly on Human–AI collaboration no matter how good the output looks; we reward teams who lead the AI, not teams who are led by it. Judges will review commit history and AI chat logs as part of verifying sustained, genuine work.

7. Final Round (21, 22 September 2026)

Top 10 teams from the initial round advance to the on-site final at SLIIT Main Auditorium. The final round format and guidelines will be shared with shortlisted teams on 18 September 2026. Finalists should expect to demonstrate their solution live, answer detailed questions from the judging panel about any part of their system, and adapt their solution to requirements announced on the day.

8. Support & Contact

- Official communication channel: hackathon@slit.lk

9. Appendix — Getting AI API Access Without Spending Money

Building for this challenge will likely require access to embedding models and large language model (LLM) APIs. This appendix lists practical, verified ways to get that access for free or nearly free.

Information verified August 2026 — providers change their free tiers frequently, so always confirm current terms on the provider's own pricing page before you rely on it.

9.0 Before You Start: Get a Card That Works Online

Most of the best free tiers require adding a debit or credit card for identity verification, even though nothing is charged if you stay within the free allowance. To unlock the options below, at least one team member should have a debit or credit card with international/online transactions enabled. If your card doesn't work on foreign websites, contact your bank and ask them to enable international online transactions — most Sri Lankan banks can do this quickly via their app or a call. A card is used for verification; you will not be charged on the paths below as long as you follow the instructions and stay within free limits. If no team member can get a card, Option 9.2 (OpenRouter free models) works with no card at all.

9.1 Embedding Models — Voyage AI (200M Free Tokens)

Voyage AI (a MongoDB company) provides some of the best embedding models available, with an unusually generous free allowance for new accounts.

1. **Sign up:** Go to the Voyage AI dashboard at dashboard.voyageai.com and sign in — you can use your existing Google account.

2. **Add a payment card:** In the billing section, add your card. This is what unlocks the free token allowance; you are not charged for it.
3. **Create an API key:** Create an API key from the dashboard and store it safely (see 9.4).
4. **What you get:** Your account gets 200 million free tokens covering the current-generation models: voyage-4-large, voyage-4, voyage-4-lite, voyage-context-4, and voyage-code-3, plus the rerank-2.5 reranker models. This is a one-time allowance per new account — 200M tokens is far more than this challenge needs if used sensibly.
5. **Avoid charges:** Only usage beyond the free allowance is billed to your card, so monitor your usage in the dashboard and stay within the free tokens. Note that older models (voyage-3.x and earlier) have no free allowance — use the 4-series models only. Useful property: all voyage-4 models share the same vector space, so you can embed documents with voyage-4-large and embed queries with the cheaper voyage-4-lite without re-indexing.

9.2 LLM Access — OpenRouter (Free Models, No Card)

OpenRouter (openrouter.ai) is a single API gateway to hundreds of models from all major providers, using one key and an OpenAI-compatible API. It is the recommended starting point for LLM access in this competition.

1. **Sign up:** Create an account at openrouter.ai with email, Google, or GitHub. No card is required. Generate an API key from the account settings.
2. **Free models:** Models whose ID ends in “:free” cost nothing and work with a zero balance — around 20–30 rotating open-weight models (Llama, Qwen, Gemma, and others). Point any OpenAI-compatible SDK at <https://openrouter.ai/api/v1> and use the free model ID. Limits: roughly 20 requests/minute and 50 requests/day on a card-free account.
3. **The \$10 upgrade:** A one-time purchase of \$10 in credits permanently raises your daily free-model cap from 50 to 1,000 requests, and the credits never expire — you can also spend them on paid models. This is the single best value upgrade if your team can afford it.
4. **Cheap paid models:** With a small credit balance (\$5–10 goes a long way), you can use highly capable, very cheap open models such as the DeepSeek series — excellent reasoning quality at a fraction of frontier-model prices. Check openrouter.ai/models for current free and low-cost options, as the lineup changes over time.

9.3 API Key Hygiene & Rate-Limit Survival

- Never commit API keys to Git, paste them in chats or screenshots, or hard-code them in source files. Load keys from environment variables or a .env file that is listed in .gitignore. Judges will see your full repository and commit history — a leaked key in an old commit stays leaked.
- If a key does leak, revoke it immediately in the provider dashboard and create a new one. Assume any key that has ever been visible anywhere is compromised.
- Free tiers rate-limit you with HTTP 429 errors. Build retry with exponential backoff (wait 1s, 2s, 4s, 8s...) into every API call from day one — systems without it fail unpredictably, including during your demo recording.

- Limits are usually applied per account or per project. Four team members creating their own individual accounts (each with their own key) is legitimate and multiplies your available quota for development.

Design to be cheap: cache responses you've already generated, keep prompts small, and test on a handful of questions rather than the whole set on every run. Frugal engineering is also good engineering — and judges notice.